

Next, we prepare a font attribute for rendering text ❸. The None argument tells Pygame to use the default font, and 48 specifies the size of the text. To center the button on the screen, we create a rect for the button ❹ and set its center attribute to match that of the screen.

Pygame works with text by rendering the string you want to display as an image. Finally, we call `_prep_msg()` to handle this rendering ❺.

Here's the code for `_prep_msg()`:

```
button.py def _prep_msg(self, msg):
    """Turn msg into a rendered image and center text on the button."""
    ❶ self.msg_image = self.font.render(msg, True, self.text_color,
        self.button_color)
    ❷ self.msg_image_rect = self.msg_image.get_rect()
    self.msg_image_rect.center = self.rect.center
```

The `_prep_msg()` method needs a `self` parameter and the text to be rendered as an image (`msg`). The call to `font.render()` turns the text stored in `msg` into an image, which we then store in `self.msg_image` ❶. The `font.render()` method also takes a Boolean value to turn antialiasing on or off (*antialiasing* makes the edges of the text smoother). The remaining arguments are the specified font color and background color. We set antialiasing to True and set the text background to the same color as the button. (If you don't include a background color, Pygame will try to render the font with a transparent background.)

We center the text image on the button by creating a rect from the image and setting its center attribute to match that of the button ❷.

Finally, we create a `draw_button()` method that we can call to display the button onscreen:

```
button.py def draw_button(self):
    """Draw blank button and then draw message."""
    self.screen.fill(self.button_color, self.rect)
    self.screen.blit(self.msg_image, self.msg_image_rect)
```

We call `screen.fill()` to draw the rectangular portion of the button. Then we call `screen.blit()` to draw the text image to the screen, passing it an image and the rect object associated with the image. This completes the Button class.

Drawing the Button to the Screen

We'll use the Button class to create a Play button in `AlienInvasion`. First, we'll update the import statements:

```
alien_invasion.py --snip--
from game_stats import GameStats
from button import Button
```
